Natural Language Understanding
Assignment - 2

Name: S Manikanta Varshit
Roll No.: B22AI038

PROBLEM 1: Learning Word Embeddings from IIT Jodhpur Data

Objective

The objective of this assignment is to learn distributed word representations (Word2Vec embeddings) from textual data collected from IIT Jodhpur sources and to analyze the semantic structure captured by these embeddings using similarity, analogy, and visualization techniques.

TASK 1: DATASET PREPARATION

Data Collection

Textual data was collected from multiple IIT Jodhpur sources, including:

- Official website pages (introduction, programs, infrastructure)
- Academic regulations
- Department pages (Computer Science & Engineering)
- Admission and campus-related pages

Preprocessing Steps

To ensure high-quality input for training, the following preprocessing steps were applied:

1. **Removal of Boilerplate Content**
    - HTML tags, scripts, navigation bars, and footers are removed.
    - Template artifacts like ((...)) and {{...}} were eliminated.

2. **Tokenization**

- A custom tokenizer was built using regular expressions.
- Implemented a word tokenizer.

3. **Lowercasing**
   - All text was converted to lowercase for consistency.
     Ex: It prevents the model from thinking "Engineer" and "engineer" are different entities.

4. **Normalization of Academic Terms**
   - Terms like *B.Tech*, *M.Tech*, *Ph.D* were normalized to: btech, mtech, phd

5. **Removal of Non-textual Content**
   - Numbers, punctuation, and special characters were removed.
   - Only alphabetic tokens were retained.

6. **Stopword Removal**
   - Common words (e.g., *the, is, and*) were removed.

Dataset Statistics

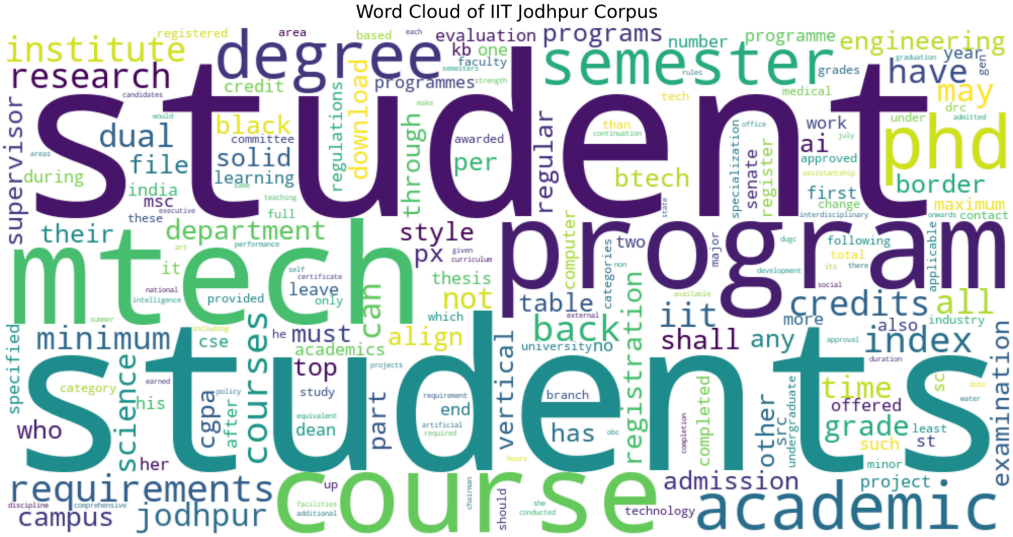
- **Number of Documents:** 17
- **Total Tokens:** 17,749
- **Vocabulary Size:** 3,126

Word Cloud

The word cloud highlights dominant academic themes such as:

- student, program, course, mtech, academic

Word Cloud of IIT Jodhpur Corpus


student
program
mtech
students
course
academic
degree
semester
phd
institute
research
credits
requirements
minimum
department
jodhpur
campus
index
grade
time
admission
examination
supervisor
science
courses
dual
file
solid
black
learning
btech
iit
shall
any
table
back
align
style
thesis
not
px
regular
registration
vertical
part
cgpa
top
must
academics
leave
provided
cse
minimum
specified
who
her
his
dean
study
end
has
branch
completed
other
src
undergraduate
offered
such
least
st
project
technology
programs
programmes
programme
engineering
evaluation
download
regulations
through
per
awarded
computer
two
major
specialization
senate
register
work
ai
approved
first
change
interdisciplinary
maximum
contact
applicable
following
total
more
also
industry
university
no
external
which
may
have
year
grades
medical
tech
number
faculty
semesters
credit
kb
one
registered
area
based
each
candidates
would
make
than
continuation
office
under
admitted
drc
july
assistantship
interdisciplinary
onwards
state
class
performance
art
given
curriculum
self
certificate
national
intelligence
executive
these
full
teaching
areas
during
india
msc
committee
learning
solid
dual
file
black
supervisor
their
it
provided
leave
only
cse
minimum
specified
category
earned
policy
after
cgpa
courses
science
his
who
her
dean
study
top
must
academics
align
back
table
not
thesis
style
px
can
part
end
has
branch
completed
other
src
undergraduate
offered
such
least
st
project
technology
admission
academic
course
requirements
campus
jodhpur
up
discipline
comprehensive
should
chairman
facilities
additional
conducted
she
hours
completion
obc
artifical
required
equivalent
requirement
projects
he
external
university
no
approval
duration
water
date
minor
least
st
such
grade
time
offered
examination
sc
applicable
industry
contact
maximum
onwards
interdisciplinary
change
total
following
more
also
any
shall
iit
registration
vertical
table
back
align
style
thesis
not
px
regular

TASK 2: MODEL TRAINING

Models Implemented

Two Word2Vec architecture is implemented **from scratch using NumPy**:

1. Continuous Bag of Words (CBOW)

- Predicts a target/centre word from its surrounding context.
- Uses the average of context vectors.
- Faster and more stable.

2. Skip-gram with Negative Sampling

- Predicts context words from a target/center word.
- Uses negative sampling to approximate softmax.
- Better for capturing semantic relationships.

Training Observations

1. Loss Behaviour

- Loss consistently decreases across epochs → model is learning.
- Skip-gram loss is significantly higher due to:
    - Multiple negative samples
    - More updates per example

2. Effect of Embedding Dimension

Config	Embedding Dimension	Window Size	Negative Samples
1	20	5	5
2	100	5	5

In Skip Gram:

Epoch	Dim=20	Dim=100
1	1,077,894	2,241,403
50	206,555	154,320

Higher dimension (**100**) starts with much higher loss due to:

- Larger parameter space
- More complex optimization

However, it converges to a **lower final loss**, indicating better representation learning

In CBOW:

Epoch	Dim=20	Dim=100
1	14,299	22,219
50	1,952	536

- CBOW benefits significantly from higher dimension
- Final loss is **4 time lower** for dimension 100
- Indicates **better predictive capability**

3. Effect of Window Size

- Window = 2 → captures local context
- Window = 10 → captures broader semantic context
- Larger window increases loss but captures **global relationships**

4. Effect of Negative Sampling

- Increasing negative samples (5 → 10):
    - Improves discrimination between words
    - Slightly increases training cost

Key Insight

> Skip-gram with moderate window size (5) and higher dimensions (100) provides the best balance between semantic quality and efficiency.

TASK 3: SEMANTIC ANALYSIS

Nearest Neighbors (Skip-gram)

research →
work (0.4660), projects (0.4567), proposal (0.4485), proposal (0.4445), encouraging (0.4427)

student →
admitted (0.5023), thesis (0.4961), dean (0.4959), sota (0.4757), should (0.4658)

phd →
mtech (0.6763), dual (0.6357), degree (0.5360), cse (0.5251), program (0.5225)

exam →
state (0.4799), departments (0.4654), disciplines (0.4482), courses (0.4468), grant (0.4427)

Analysis

Good Semantic Capture

- *phd → mtech, degree, program*
  → correctly captures **academic hierarchy**

- *research → projects, work*
  → reflects **research ecosystem**

Noise Observed

- Some unrelated words appear:
  - *exam → state, visiting*

- Caused by:
  - small dataset size
  - limited co-occurrence diversity

Analogy Experiments

Results:

1. **UG : BTech :: PG → [total, msc, degree]**

   - Partially correct (*msc, degree*)
   - Shows understanding of **degree progression**

2. **student : professor :: phd → [dual, associate, features]**

   - Weak mapping

    - Indicates limited relational learning

3. **exam : test :: assignment → [applicant, case, platform]**

    - Less meaningful
    - Dataset lacks strong evaluation context diversity

Word embeddings successfully capture **semantic similarity**, but analogy performance is weaker due to limited corpus size and domain constraints.

TASK 4: VISUALIZATION

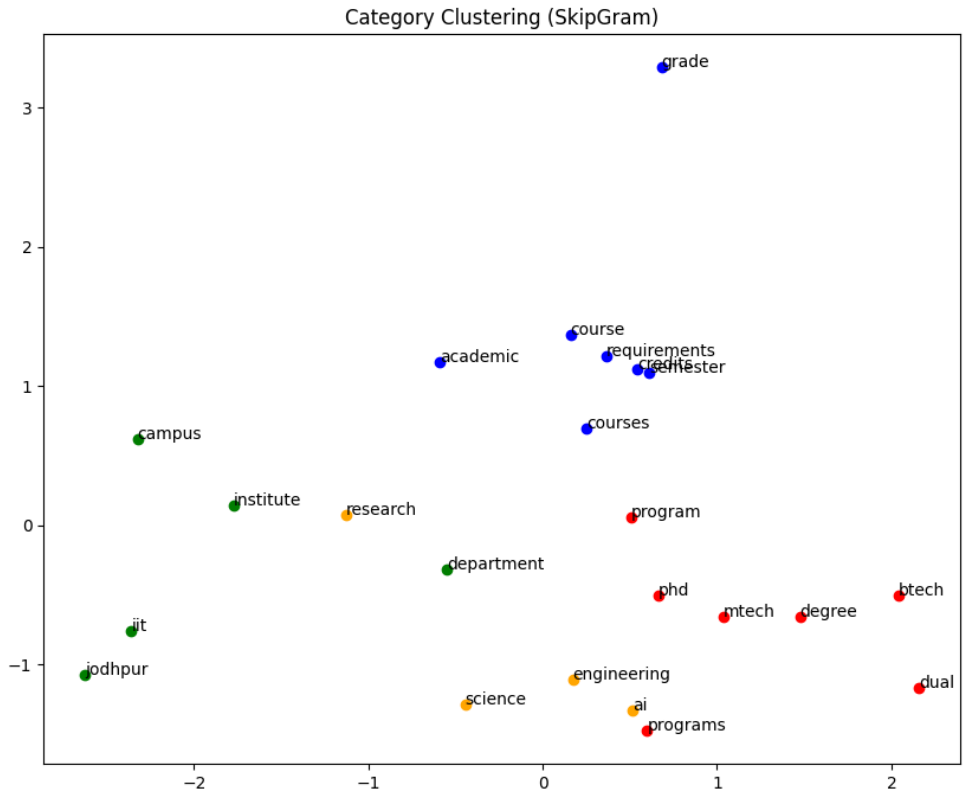
Methods Used

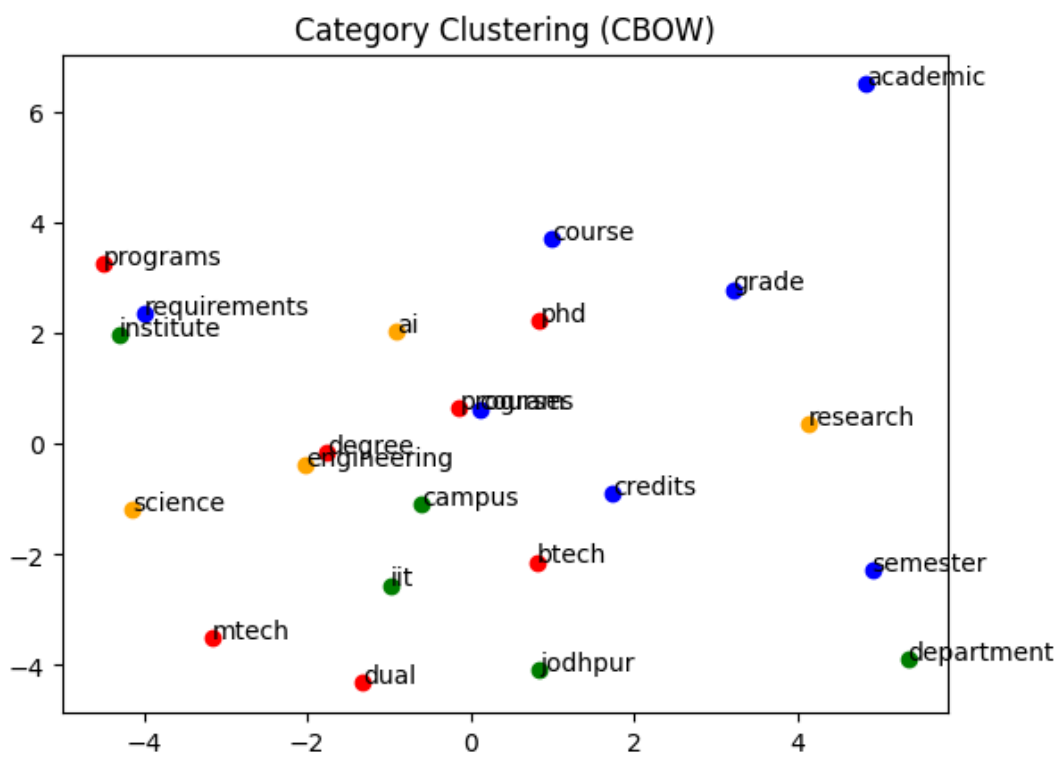
- **PCA** is used

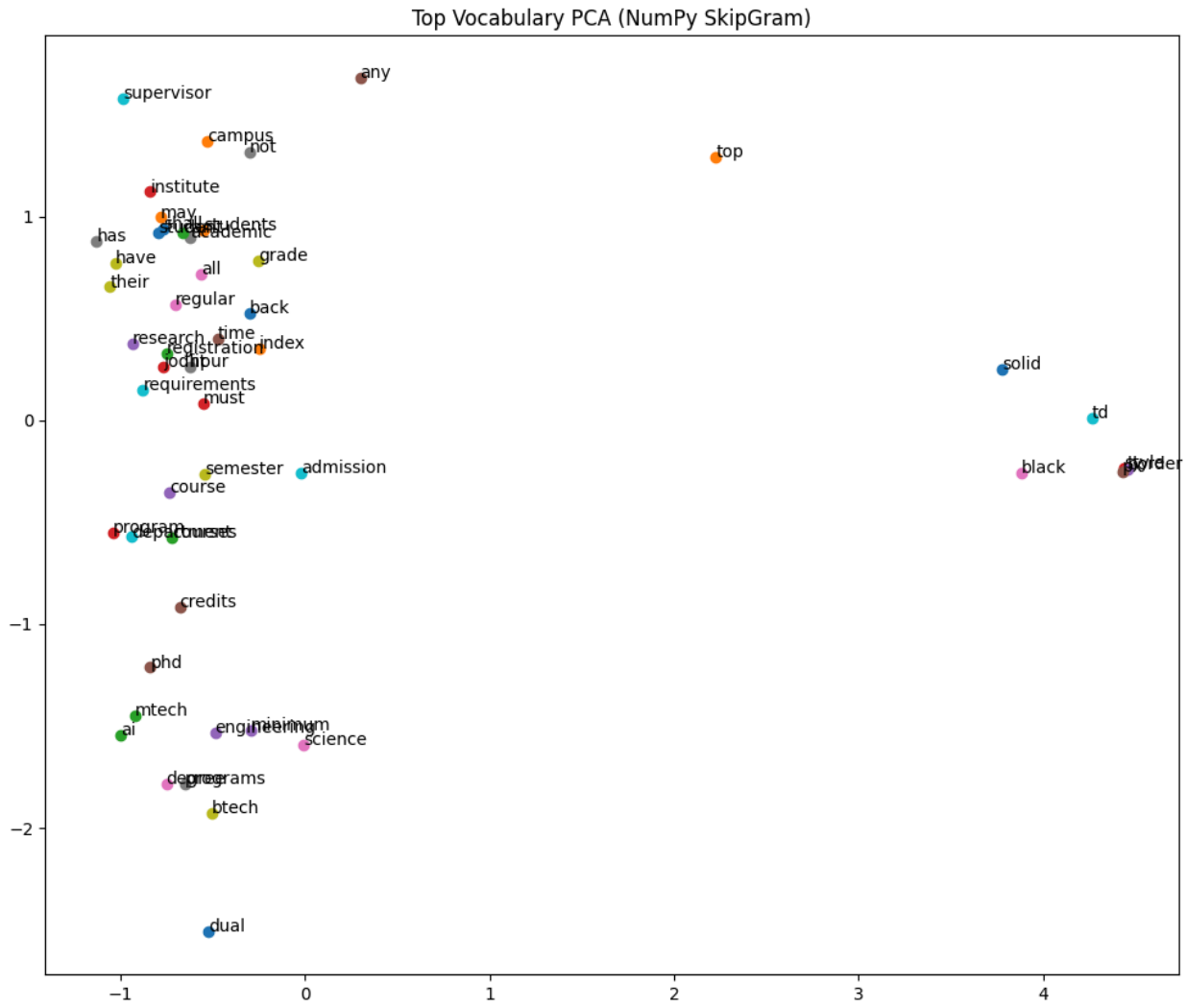
Category-Based Clustering

Words were grouped into semantic categories:

- Programs → *btech, mtech, phd*
- Academics → *course, semester, credits*
- Institute → *department, campus, iit*
- Research → *science, engineering, ai*


Category Clustering (SkipGram)
grade
course
requirements
credits
semester
academic
courses
campus
institute
research
program
department
phd
btech
mtech
degree
iit
iodhpur
engineering
dual
science
ai
programs


Category Clustering (CBOW)
academic
course
programs
grade
requirements
institute
ai
phd
research
degree
engineering
credits
science
campus
btech
semester
iit
mtech
department
jodhpur
dual



Observations

1. Clear Semantic Clusters

- Academic words group together
- Program-related words cluster tightly
- Institutional words form distinct regions

2. Skip-gram vs CBOW

Aspect	Skip-Gram	CBOW
Clarity	High	Moderate

Seperation	Strong	Compact
Semantic quality	Better	Smoother

Skip-gram embeddings produce more meaningful clusters, especially for domain-specific terms.

PROBLEM 2: CHARACTER-LEVEL NAME GENERATION USING RNN VARIANTS

Introduction

The objective of this assignment is to design and compare sequence models for **character-level name generation** using recurrent neural architectures. Given a dataset of Indian names, the task is to train models that can generate realistic and diverse new names character by character.

Three models are implemented and compared:

- Vanilla Recurrent Neural Network (RNN) (implemented from scratch)
- Bidirectional Long Short-Term Memory (BLSTM) (used inbuilt PyTorch module)
- RNN with Attention Mechanism (implemented from scratch)

TASK-0: DATASET

A dataset of **1000 Indian names** was generated and stored in TrainingNames.txt. Here, Gemini model were used.

Preprocessing Steps

- Added special tokens:
    - <SOS>: Start of sequence
    - <EOS>: End of sequence
    - <PAD>: Padding for equal length
- Built a character-level vocabulary
- Converted each name into a sequence of indices
- Applied padding to ensure uniform sequence length

TASK-1: MODEL IMPLEMENTATION

1. Vanilla RNN (From Scratch)

Architecture

The Vanilla RNN processes sequences step-by-step using the recurrence:

$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$

- **Embedding Layer**: Maps characters to dense vectors
- **Recurrent Layer**: Manually implemented using linear transformations
- **Fully Connected Layer**: Predicts next character

Key Characteristics

- Simple and computationally efficient
- Struggles with long-range dependencies

Hyperparameters

- Embedding Dimension: 64
- Hidden Size: 128
- Learning Rate: 0.001
- Epochs: 100

Trainable Parameters = 30236

2. Bidirectional LSTM (BLSTM)

Architecture

BLSTM processes the sequence in both directions:

- Forward LSTM (left → right)
- Backward LSTM (right → left)

Outputs from both directions are concatenated before prediction.

Key Characteristics

- Captures both past and future context
- Handles long-term dependencies effectively
- Higher computational cost

Hyperparameters

- Embedding Dimension: 32
- Hidden Size: 128
- Learning Rate: 0.001
- Epochs: 10
- Bidirectional: Yes & No

Trainable Parameters = 87452 (Bidirectional = No) & 157148 (Bidirectional = Yes)

3. RNN with Attention (From Scratch)

Architecture

This model extends the Vanilla RNN by incorporating an attention mechanism.

- Hidden states for all timesteps are stored
- Attention weights are computed dynamically
- A context vector is formed using weighted sum of hidden states
- The context vector is combined with the current hidden state

Attention Mechanism

$$\alpha_t = \mathrm{softmax}(score(h_t, H))$$
$$context = \sum \alpha_t h_t$$

Key Characteristics

- Focuses on relevant parts of the sequence
- Improves long-range dependency handling
- More expressive than Vanilla RNN

Hyperparameters

- Embedding Dimension: 16
- Hidden Size: 128
- Learning Rate: 0.001
- Epochs: 10
- Attention Type: Additive

Trainable Parameters = 42844

TASK-2: QUANTITATIVE EVALUATION

1. Novelty Rate

Percentage of generated names not present in the training dataset:

$$\text{Novelty} = \frac{\text{New Names}}{\text{Total Generated Names}}$$

2. Diversity

Ratio of unique generated names:

$$\text{Diversity} = \frac{\text{Unique Names}}{\text{Total Generated Names}}$$

Results

Model	**Novelty**	**Diversity**
Vanilla RNN	0.607	0.877
BLSTM (Bidirectional = False)	0.988	0.933
BLSTM (Bidirectional = True)	1.000	1.000
RNN with Attention	0.986	0.995

Analysis

1. **Vanilla RNN**:
    - Moderate novelty (~60%)
    - Good diversity (~87%)
    - Indicates partial memorization and repetition
2. **BLSTM (Bidirectional = False) OR Unidirectional:**
    - Very high novelty (~98.8%)
    - High diversity (~99.3%)
    - Generates varied outputs but may lose realism
3. **BLSTM**
    - Perfect novelty (100%) & Perfect diversity (100%)
    - The resulting names are random and don’t represent a real Indian name.
4. **RNN + Attention**:
    - High novelty (~98.7%)
    - Very high diversity (~99.5%)
    - Balanced performance compared to LSTM
    - The results give somewhat meaningful Indian names.

TASK-3 QUALITATIVE ANALYSIS

Vanilla RNN

- zarika
- tara
- nilambari
- hamindi
- pavan
- niti
- ima
- ranavar
- lakshmishi
- darshini

LSTM (Unidirectional)

- nameka
- shuunedhan
- wanukhwasha
- tarikh
- goiti
- ramdhi
- sikla
- kutetn
- myal
- nimali

BLSTM

- swokumoohesre
- mombhupikiwtt
- tivhuzocl
- ffmnaghwalus
- zugqttisht
- fagukhsev
- umpuadyu
- ucrikkuqsamwi
- pubyepaochh
- ooulkuzfni

RNN + Attention

- sudeeoth

- goni
- rogi
- udhehanh
- prrajkan
- chamyali
- venashit
- manuk
- mamani
- sridhar

Realism Analysis

- **Vanilla RNN**:
  - Mostly realistic names (tara, pavan) are generated

- **LSTM**:
  - Produces structured names
  - Some resemble Indian phonetics

- **BLSTM**:
  - Poor realism despite perfect metrics
  - Outputs is random

- **RNN + Attention**:
  - Best balance of realism and variation
  - Names like sridhar, goni, manuk resemble real patterns

Important Pointer:

As observed in the results, the **BLSTM performs poorly during the generation phase**. Because a BLSTM relies on a backward pass that reads from right to left, it is against the autoregressive generation. During training, it "cheats" by looking at the backward hidden state. During generation, those future characters do not exist yet, causing the generation process to collapse. This highlights that while **BLSTMs are excellent for full-sequence classification tasks** (like sentiment analysis), they are not great for left-to-right generative tasks.